PHASE 13 OF 18

Collections I: List, Set, Queue & Sorting

Why collections, the hierarchy, ArrayList/LinkedList/Vector/Stack internals, immutable lists, iterators and fail-fast behaviour, HashSet/LinkedHashSet/TreeSet, Queue/Deque/PriorityQueue, Comparable vs Comparator, and the Collections utility class

IN THIS PHASE

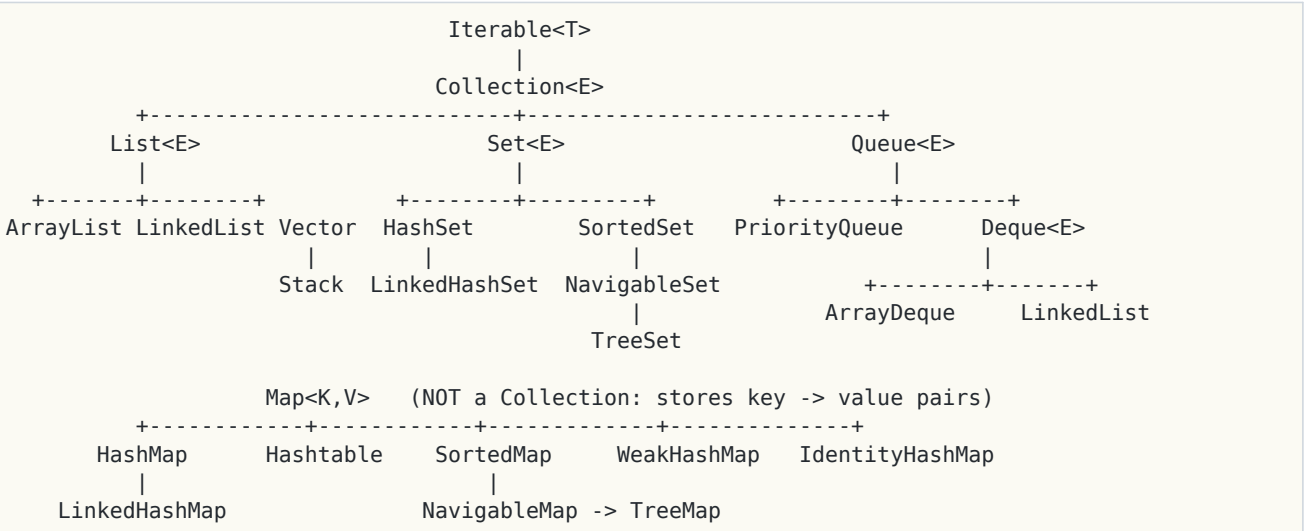
13.1 Why Collections?
13.2 The Collection Hierarchy
13.3 List: ArrayList
13.4 List: LinkedList, Vector and Stack
13.5 Iterating Collections
13.6 Set: HashSet, LinkedHashSet, TreeSet
13.7 Queue and Deque
13.8 Comparable vs Comparator
13.9 The Collections Utility Class
13.10 Predict the Output
13.11 Interview Questions
13.12 Quick Revision

13.1 Why Collections?

Arrays have limitations: **fixed size**, no built-in add/remove/search/sort operations, and shifting is up to you. The **Java Collections Framework** (JCF, `java.util`, Java 1.2) provides **interfaces** (List, Set, Queue, Map) with **ready-made implementations** and **algorithms** (sorting, searching).

Array	Collection
Fixed size	Grows and shrinks dynamically
Primitives and objects	Objects only (primitives are autoboxed)
`length` field, manual algorithms	Rich API: `add`, `remove`, `contains`, `sort`, streams
Homogeneous by declared type	Generic, type-safe (`List<String>`)
Slightly faster, less memory	Small overhead; many optimised data structures to choose from

13.2 The Collection Hierarchy



Interface	Ordering	Duplicates	Key idea
`List`	Insertion order, **index-based**	Allowed	Sequence: access by position
`Set`	Depends (none / insertion / sorted)	**Not allowed**	Uniqueness by `equals`/`hashCode` (or `compareTo`)
`Queue` / `Deque`	FIFO, priority, or both ends	Allowed	Processing order: offer, poll, peek
`Map`	Depends	Keys unique, values may repeat	Lookup by key

MODERN JAVA

Java 21 added **sequenced collections**: `SequencedCollection` (`getFirst()`, `getLast()`, `addFirst()`, `reversed()`, ...) implemented by `List`, `Deque` and `LinkedHashSet`, plus `SequencedMap` for `LinkedHashMap` and `TreeMap` (covered in Phase 14).

Core Collection methods

Method	Purpose
`add(e)`, `addAll(c)`	Insert (returns `false` if a Set already contains it)
`remove(o)`, `removeAll(c)`, `retainAll(c)`, `removeIf(pred)`, `clear()`	Remove
`contains(o)`, `containsAll(c)`	Membership (uses `equals`)
`size()`, `isEmpty()`	Counting
`iterator()`, `forEach(action)`	Traversal
`toArray()`, `toArray(new T[0])`, `toArray(String[]::new)` (11)	Convert to an array
`stream()`, `parallelStream()`	Stream pipelines (Phase 15)

13.3 List: ArrayList

`ArrayList` is a **resizable array**. It is the default choice for lists.

- Ordered by insertion; index-based; **duplicates** and **multiple nulls** allowed.
- **Fast random access** `get(i)`: O(1). Adding at the end: amortised O(1).
- Inserting or removing in the **middle** is O(n), because elements must shift.
- Not synchronized.

```
List<String> names = new ArrayList<>();       // program to the interface
names.add("Ravi");                            // [Ravi]
names.add("Amit");                            // [Ravi, Amit]
names.add(0, "Zoya");                         // [Zoya, Ravi, Amit]  (shift right)
names.set(1, "Rohit");                        // [Zoya, Rohit, Amit] (replace)
names.get(2);                                 // Amit
names.indexOf("Amit");                        // 2  (-1 if absent; uses equals)
names.contains("Zoya");                       // true
names.remove("Zoya");                         // by object -> [Rohit, Amit]
names.remove(0);                              // by INDEX  -> [Amit]
names.addAll(List.of("B", "C"));              // [Amit, B, C]
List<String> sub = names.subList(1, 3);       // [B, C]: a VIEW (changes write through)
```

Internal working

```
new ArrayList<>()       -> elementData = {} (empty; the array is allocated lazily)
first add()             -> capacity 10:   [A][ ][ ][ ][ ][ ][ ][ ][ ][ ]
11th add()              -> grow by 50%: newCapacity = old + (old >> 1) = 15
                           copy everything with Arrays.copyOf -> O(n), but rare
add(0, X) / remove(0)   -> System.arraycopy shifts every later element: O(n)
```

Constructor	Purpose
`ArrayList()`	Empty; capacity 10 allocated on the first add
`ArrayList(int initialCapacity)`	Pre-size when you know the approximate count (avoids repeated growth)
`ArrayList(Collection c)`	Copy another collection: `new ArrayList<>(set)`

TIP

`size()` is the number of elements; **capacity** is the internal array length, which you can't query. `ensureCapacity(n)` and `trimToSize()` manage it. Prefer `List<String> x = new ArrayList<>()` (interface type) so implementations can be swapped easily.

13.4 List: LinkedList, Vector and Stack

LinkedList

A **doubly linked list**: each node holds `prev`, `item` and `next`. It implements both **List** and **Deque**.

```
 first                                                  last
   |                                                     |
   v                                                     v
[null|Ravi|*] <--> [*|Amit|*] <--> [*|Zoya|null]
```

```
LinkedList<Integer> ll = new LinkedList<>(List.of(20, 30));
ll.addFirst(10);  ll.addLast(40);          // O(1) at the ends: [10, 20, 30, 40]
ll.removeFirst(); ll.peekLast();           // Deque operations
ll.get(2);                                 // O(n): walks from the nearer end
```

NOTE

LinkedList's middle insertion is O(1) **only once you're at the node** (with a `ListIterator`). Finding the position is O(n). Its nodes are scattered in memory and each carries 2 extra references, so in practice **ArrayList is faster for almost everything**, and `ArrayDeque` is faster for queues and stacks.

Vector and Stack (legacy)

- `Vector`: a synchronized growable array (Java 1.0). It **doubles** its capacity when full.
- `Stack`: extends `Vector`, LIFO: `push`, `pop`, `peek`, `empty`, `search`.

Both are **legacy**. Use `ArrayList` (with external synchronization or concurrent collections when needed) and `ArrayDeque` for stacks.

```
Deque<Integer> stack = new ArrayDeque<>();   // preferred stack
stack.push(1); stack.push(2); stack.push(3);
stack.pop();     // 3   (LIFO)
stack.peek();    // 2
```

ArrayList vs LinkedList vs Vector vs Stack

Feature	ArrayList	LinkedList	Vector	Stack
Structure	Resizable array	Doubly linked list	Resizable array	Vector subclass
`get(i)`	**O(1)**	O(n)	O(1)	O(1)
Add/remove at end	Amortised O(1)	O(1)	O(1)	O(1) push/pop
Add/remove at start or middle	O(n) shift	O(1) once positioned (finding it is O(n))	O(n)	n/a
Memory	Compact	Extra 2 refs per node	Compact	Compact
Growth	+50%	n/a (per node)	×2	×2
Thread-safe	No	No	Yes (synchronized)	Yes
Interfaces	List, RandomAccess	List, **Deque**	List, RandomAccess	List (LIFO methods)
Status	**Default choice**	Rarely the best	Legacy	Legacy → use `ArrayDeque`

Unmodifiable and immutable lists

Creation	Add/remove	Set an element	Nulls	Backed by the source?
`List.of(a, b)` (9)	✗ UOE	✗ UOE	✗ NPE	No: truly immutable
`List.copyOf(coll)` (10)	✗	✗	✗	No: an immutable copy
`Arrays.asList(a, b)`	✗ UOE	✓ (writes through to the array)	✓	**Yes**: a fixed-size view of the array
`Collections.unmodifiableList(list)`	✗	✗	✓	**Yes**: a read-only *view*; the original can still change
`stream().toList()` (16)	✗	✗	✓	No

(UOE = `UnsupportedOperationException`.)

13.5 Iterating Collections

```
List<String> list = new ArrayList<>(List.of("A", "B", "C"));

for (int i = 0; i < list.size(); i++) System.out.print(list.get(i));   // index loop (Lists)
for (String s : list) System.out.print(s);                             // for-each
Iterator<String> it = list.iterator();                                 // explicit Iterator
while (it.hasNext()) System.out.print(it.next());
list.forEach(s -> System.out.print(s));                                // Java 8 forEach
list.forEach(System.out::print);                                       // method reference
list.stream().filter(s -> !s.equals("B")).forEach(System.out::print); // streams

ListIterator<String> li = list.listIterator(list.size());              // start at the end
while (li.hasPrevious()) System.out.print(li.previous());              // CBA (backwards)
```

Cursors: Iterator vs ListIterator vs Enumeration

Feature	Enumeration (legacy)	Iterator	ListIterator
Applies to	Vector, Hashtable	**All** collections	List only
Direction	Forward	Forward	**Forward and backward**
Methods	hasMoreElements, nextElement	hasNext, next, remove	+ hasPrevious, previous, nextIndex, add, set
Modify while iterating	No	Remove	Remove, **add**, **set**
Obtained via	vector.elements()	coll.iterator()	list.listIterator()

Fail-fast iterators and ConcurrentModificationException

Most java.util collections (ArrayList, HashMap, HashSet...) have **fail-fast** iterators. Each collection keeps a modCount; if the collection is **structurally modified** during iteration by anything other than the iterator itself, the next next() call throws ConcurrentModificationException (CME). This happens even in a **single** thread.

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6));

for (Integer n : nums) {
    if (n % 2 == 0) nums.remove(n);          // ConcurrentModificationException!
}

Iterator<Integer> it = nums.iterator();      // FIX 1: remove via the iterator
while (it.hasNext()) {
    if (it.next() % 2 == 0) it.remove();
}
nums.removeIf(n -> n % 2 == 0);              // FIX 2 (Java 8): simplest
List<Integer> odds = nums.stream()           // FIX 3: build a new list
        .filter(n -> n % 2 != 0).toList();
for (Integer n : new ArrayList<>(nums)) { }  // FIX 4: iterate over a copy
```

Fail-safe (more precisely, *snapshot* or *weakly consistent*) iterators come from concurrent collections. CopyOnWriteArrayList iterates over a **snapshot** taken when the iterator was created; ConcurrentHashMap iterators are **weakly consistent**: they never throw CME and may or may not reflect later changes.

```
List<String> cow = new CopyOnWriteArrayList<>(List.of("A", "B"));
for (String s : cow) cow.add("X");     // no CME: the loop sees only the snapshot [A, B]
System.out.println(cow);               // [A, B, X, X]
```

	Fail-fast	Snapshot / weakly consistent ("fail-safe")
Collections	ArrayList, LinkedList, HashMap, HashSet, TreeMap	CopyOnWriteArrayList/Set, ConcurrentHashMap, ConcurrentLinkedQueue
On concurrent modification	Throws CME (best effort)	No exception
Sees changes made during iteration	n/a	Snapshot: never; weakly consistent: maybe
Memory / cost	No extra cost	Copy-on-write cost, or concurrent design

13.6 Set: HashSet, LinkedHashSet, TreeSet

A `Set` stores **unique** elements, so `add` returns `false` for a duplicate.

```
Set<String> hs = new HashSet<>(List.of("Pune", "Delhi", "Mumbai"));
hs.add("Delhi");                    // false: already present
hs.add(null);                       // one null allowed
System.out.println(hs);             // order NOT guaranteed, e.g. [null, Delhi, Pune, Mumbai]

Set<String> lhs = new LinkedHashSet<>(List.of("Pune", "Delhi", "Mumbai"));
System.out.println(lhs);            // [Pune, Delhi, Mumbai]  insertion order

Set<String> ts = new TreeSet<>(List.of("Pune", "Delhi", "Mumbai"));
System.out.println(ts);             // [Delhi, Mumbai, Pune]  sorted
// ts.add(null);                    // NullPointerException
```

Feature	HashSet	LinkedHashSet	TreeSet
Backed by	`HashMap` (elements stored as keys)	`LinkedHashMap`	`TreeMap` (red-black tree)
Order	None (hash order)	**Insertion** order	**Sorted** (natural or Comparator)
`add` / `contains` / `remove`	O(1) average	O(1) average	**O(log n)**
Null	One null	One null	**No** (NPE)
Uniqueness decided by	`hashCode()` + `equals()`	`hashCode()` + `equals()`	`compareTo()`/`compare()` **== 0**
Extra interfaces	n/a	`SequencedSet` (21)	`NavigableSet`, `SortedSet`

PITFALL

Custom objects in a `HashSet` **must** override `equals()` **and** `hashCode()` (Phase 8), otherwise "duplicates" get in. In a `TreeSet` they must be `Comparable` (or you must supply a `Comparator`), otherwise you get a `ClassCastException` on the first add. A `TreeSet` treats elements with `compare == 0` as **duplicates**, even if `equals` says they differ.

TreeSet navigation (NavigableSet)

```
TreeSet<Integer> scores = new TreeSet<>(List.of(45, 72, 88, 91, 60));   // [45, 60, 72, 88, 91]
scores.first();        // 45            scores.last();          // 91
scores.floor(70);      // 60  (<= 70)   scores.ceiling(70);     // 72 (>= 70)
scores.lower(72);      // 60  (< 72)    scores.higher(72);      // 88 (> 72)
scores.headSet(72);    // [45, 60]      (< 72)
scores.tailSet(72);    // [72, 88, 91]  (>= 72)
scores.subSet(60, 90); // [60, 72, 88]  [60, 90)
scores.descendingSet();// [91, 88, 72, 60, 45]
scores.pollFirst();    // removes and returns 45
```

Set algebra

```
Set<Integer> a = new HashSet<>(Set.of(1, 2, 3, 4));
Set<Integer> b = Set.of(3, 4, 5);
Set<Integer> union = new HashSet<>(a);        union.addAll(b);       // [1, 2, 3, 4, 5]
Set<Integer> inter = new HashSet<>(a);        inter.retainAll(b);    // [3, 4]
Set<Integer> diff  = new HashSet<>(a);        diff.removeAll(b);     // [1, 2]
Collections.disjoint(a, Set.of(9));           // true: no common elements

List<String> dupes = List.of("a", "b", "a", "c", "b");
List<String> unique = new ArrayList<>(new LinkedHashSet<>(dupes));   // [a, b, c] order kept
```

13.7 Queue and Deque

Operation	Throws an exception on failure	Returns a special value
Insert	`add(e)`: `IllegalStateException` if full	`offer(e)`: `false`
Remove head	`remove()`: `NoSuchElementException`	`poll()`: `null`
Examine head	`element()`: `NoSuchElementException`	`peek()`: `null`

```
Queue<String> q = new ArrayDeque<>();      // FIFO
q.offer("T1"); q.offer("T2"); q.offer("T3");
q.peek();      // T1 (not removed)
q.poll();      // T1 (removed)
q.poll(); q.poll();
q.poll();      // null (empty)
// q.remove(); // NoSuchElementException

Deque<String> dq = new ArrayDeque<>();     // double-ended: queue AND stack
dq.offerFirst("B"); dq.offerFirst("A"); dq.offerLast("C");   // [A, B, C]
dq.pollFirst(); dq.pollLast();                               // A, C
```

TIP

`ArrayDeque` is the recommended general-purpose **queue and stack**: faster than `LinkedList` and `Stack`, resizable, and with no capacity limit. It does **not** accept `null`.

PriorityQueue: a binary heap

Elements come out in **priority order** (natural order = smallest first, a min-heap), not insertion order.

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
for (int x : new int[]{50, 20, 40, 10, 30}) pq.offer(x);
System.out.println(pq);            // [10, 20, 40, 50, 30]  heap layout, NOT sorted order!
while (!pq.isEmpty()) System.out.print(pq.poll() + " ");   // 10 20 30 40 50 (sorted removal)

PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());

record Txn(String id, int amount) { }
PriorityQueue<Txn> highValueFirst =
        new PriorityQueue<>(Comparator.comparingInt(Txn::amount).reversed());
highValueFirst.add(new Txn("T1", 500));
highValueFirst.add(new Txn("T2", 90_000));
highValueFirst.poll();             // Txn[id=T2, amount=90000] -> review big ones first
```

	Complexity
`offer` / `poll`	O(log n)
`peek`	O(1)
`contains` / `remove(Object)`	O(n)

PITFALL

Printing or iterating a `PriorityQueue` does **not** show sorted order. Only repeated `poll()` returns elements in priority order. `PriorityQueue` doesn't allow `null` and isn't thread-safe (use `PriorityBlockingQueue`).

13.8 Comparable vs Comparator

Comparable: natural ordering, defined inside the class

```
public class Employee implements Comparable<Employee> {
    private final int id; private final String name; private final double salary;
    public Employee(int id, String name, double salary) {
        this.id = id; this.name = name; this.salary = salary;
    }
    public int getId() { return id; }
    public String getName() { return name; }
    public double getSalary() { return salary; }

    @Override
    public int compareTo(Employee other) {           // natural order: by id
        return Integer.compare(this.id, other.id);   // negative / 0 / positive
    }
    @Override public String toString() { return id + ":" + name + ":" + salary; }
}

List<Employee> emps = new ArrayList<>(List.of(
        new Employee(3, "Ravi", 50000), new Employee(1, "Zoya", 70000),
        new Employee(2, "Amit", 50000)));
Collections.sort(emps);          // uses compareTo -> by id: 1:Zoya, 2:Amit, 3:Ravi
```

Comparator: external, any number of orderings

```
emps.sort(Comparator.comparing(Employee::getName));                    // by name
emps.sort(Comparator.comparingDouble(Employee::getSalary).reversed()); // salary desc
emps.sort(Comparator.comparingDouble(Employee::getSalary)              // salary asc,
        .thenComparing(Employee::getName));                            // then name
emps.sort((a, b) -> a.getName().compareToIgnoreCase(b.getName()));    // lambda

Comparator<Employee> byNameNullsLast = Comparator.comparing(
        Employee::getName, Comparator.nullsLast(Comparator.naturalOrder()));

Comparator<Employee> legacy = new Comparator<>() {                     // pre-Java 8 style
    @Override public int compare(Employee a, Employee b) {
        return Double.compare(a.getSalary(), b.getSalary());
    }
};
TreeSet<Employee> bySalary = new TreeSet<>(legacy);   // sorted collections accept comparators
```

	Comparable<T>	Comparator<T>
Package	java.lang	java.util
Method	int compareTo(T other)	int compare(T a, T b)
Defined	**Inside** the class being sorted	**Outside**: a separate class, lambda or method reference
How many orderings	One ("natural" ordering)	Unlimited
Modifying the class	Required	Not required (works for classes you don't own)
Used by	Collections.sort(list), TreeSet, TreeMap, Arrays.sort	list.sort(cmp), new TreeSet<>(cmp), stream.sorted(cmp)
Examples	String, Integer, LocalDate, BigDecimal	Comparator.comparing(...), String.CASE_INSENSITIVE_ORDER

PITFALL

Don't write comparators as `return a.id - b.id;`. Subtraction can **overflow** for large or negative values and give the wrong sign. Use `Integer.compare(a, b)`, `Long.compare` or `Comparator.comparingInt`.

> **NOTE**
> Try to keep `compareTo` **consistent with equals** (`compareTo == 0` exactly when `equals` is true). `BigDecimal` is a known exception: `new BigDecimal("2.0")` and `"2.00"` compare equal but aren't `equals`, so a `TreeSet` keeps one of them and a `HashSet` keeps both.

13.9 The Collections Utility Class

`java.util.Collections` (note the **s**) contains static algorithms and wrappers. `Collection` (no s) is the **interface**.

Method	Purpose
`sort(list)` / `sort(list, cmp)`	Stable sort (TimSort), O(n log n)
`reverse(list)`, `shuffle(list)`, `swap(list, i, j)`, `rotate(list, d)`	Reorder
`max(coll)`, `min(coll)` (optionally with a comparator)	Extremes
`frequency(coll, o)`	Count occurrences
`binarySearch(list, key)`	O(log n) search on a **sorted** list
`reverseOrder()` / `reverseOrder(cmp)`	Descending comparator
`fill(list, v)`, `nCopies(n, v)`, `addAll(coll, a, b, c)`	Bulk helpers
`unmodifiableList/Set/Map(...)`	Read-only views
`synchronizedList/Set/Map(...)`	Thread-safe wrappers (lock around each call)
`emptyList()`, `singletonList(x)`, `disjoint(c1, c2)`	Small helpers

```
List<Integer> marks = new ArrayList<>(List.of(70, 95, 82, 95, 60));
Collections.sort(marks);                              // [60, 70, 82, 95, 95]
Collections.reverse(marks);                           // [95, 95, 82, 70, 60]
Collections.max(marks);                               // 95
Collections.frequency(marks, 95);                     // 2
Collections.sort(marks);
Collections.binarySearch(marks, 82);                  // 2
marks.sort(Collections.reverseOrder());               // [95, 95, 82, 70, 60]

List<String> safe = Collections.synchronizedList(new ArrayList<>());
synchronized (safe) {                                 // iteration still needs MANUAL locking
    for (String s : safe) { }
}
```

13.10 Predict the Output

```
List<Integer> l = new ArrayList<>(List.of(10, 20, 30));
l.remove(1);                        // removes the element AT INDEX 1
l.remove(Integer.valueOf(10));      // removes the VALUE 10
System.out.println(l);              // [30]

Set<String> s = new TreeSet<>(List.of("b", "B", "a"));
System.out.println(s);              // [B, a, b]  (uppercase sorts first)

Set<String> ci = new TreeSet<>(String.CASE_INSENSITIVE_ORDER);
ci.addAll(List.of("b", "B", "a"));
System.out.println(ci);             // [a, b]  ("B" equals "b" by compare: a duplicate)

Queue<Integer> pq = new PriorityQueue<>(List.of(5, 1, 3));
System.out.println(pq.poll() + " " + pq.peek());   // 1 3

Deque<Integer> st = new ArrayDeque<>();
st.push(1); st.push(2); st.push(3);
System.out.println(st.pop() + " " + st);           // 3 [2, 1]

List<String> fixed = Arrays.asList("x", "y");
fixed.set(0, "z");
System.out.println(fixed);                          // [z, y]
try { fixed.add("w"); }
catch (UnsupportedOperationException e) { System.out.println("UOE"); }   // UOE
```

13.11 Interview Questions

Q. What is the difference between Collection and Collections?

`Collection` is the root interface of List, Set and Queue. `Collections` is a utility class of static methods: sort, reverse, max, unmodifiable and synchronized wrappers.

Q. What is the difference between ArrayList and LinkedList?

ArrayList is a resizable array: O(1) random access, O(n) middle insertions or removals, compact memory. LinkedList is a doubly linked list: O(n) access, O(1) insert or remove at the ends or at an iterator position, more memory. ArrayList is the better default.

Q. How does ArrayList grow?

The default capacity of 10 is allocated on the first add. When full, a new array about 1.5× the size (`old + old/2`) is created and the elements are copied.

Q. What is the difference between ArrayList and Vector?

Vector is synchronized (thread-safe but slower), legacy, and doubles in size. ArrayList is unsynchronized and grows by 50%. Use concurrent collections when you need thread safety.

Q. What is the difference between Iterator and ListIterator?

Iterator works with any collection, forward only, and can remove. ListIterator is List-only, moves both directions, and can also `add`, `set` and report indexes.

Q. What is ConcurrentModificationException? How do you avoid it?

It is thrown by fail-fast iterators when a collection is structurally modified during iteration by anything other than the iterator. Avoid it with `iterator.remove()`, `removeIf()`, collecting to a new list, iterating over a copy, or concurrent collections.

Q. What is the difference between fail-fast and fail-safe iterators?

Fail-fast iterators (ArrayList, HashMap) throw CME on concurrent modification. Fail-safe iterators (CopyOnWriteArrayList's snapshot, ConcurrentHashMap's weakly consistent iterators) don't throw, and work on a snapshot or a best-effort live view.

Q. How does HashSet ensure uniqueness?

It is backed by a HashMap: elements are stored as keys with a dummy value, and uniqueness is decided by `hashCode()` then `equals()`.

Q. What is the difference between HashSet, LinkedHashSet and TreeSet?
No order with O(1) operations; insertion order with O(1) operations; sorted order with O(log n) operations. HashSet and LinkedHashSet allow one null; TreeSet doesn't allow null and uses `compareTo`/`compare` for uniqueness.

Q. What is the difference between Comparable and Comparator?
Comparable defines one natural ordering inside the class (`compareTo`). Comparator defines any number of external orderings (`compare`), and works for classes you can't modify. Java 8 adds `comparing`, `thenComparing` and `reversed`.

Q. What is the difference between `poll()` and `remove()` on a Queue?
Both remove the head. `poll()` returns `null` if the queue is empty; `remove()` throws `NoSuchElementException`. The same pattern holds for `peek()`/`element()` and `offer()`/`add()`.

Q. Why prefer ArrayDeque over Stack?
Stack is a legacy synchronized Vector subclass. ArrayDeque is faster, unsynchronized, and offers both stack (`push`/`pop`) and queue operations through the Deque interface.

Q. What is the difference between `List.of`, `Arrays.asList` and `Collections.unmodifiableList`?
`List.of` is truly immutable and rejects nulls. `Arrays.asList` is a fixed-size view of an array: `set` works and writes through, but add/remove don't. `unmodifiableList` is a read-only view of another list that still changes if the original changes.

13.12 Quick Revision

- JCF = interfaces (List, Set, Queue/Deque, Map) + implementations + algorithms. Map is **not** a Collection.
- **List**: ordered, indexed, duplicates. ArrayList (default, O(1) get, +50% growth); LinkedList (also a Deque); Vector/Stack are legacy.
- Immutable: `List.of`/`copyOf` (no nulls). `Arrays.asList` is fixed-size and writes through. `unmodifiableList` is a view.
- Iteration: for, for-each, Iterator, ListIterator (both directions; add/set), `forEach`, streams.
- Fail-fast → CME. Fix with `it.remove()`, `removeIf`, a copy, or concurrent collections (snapshot / weakly consistent).
- **Set**: unique. HashSet (no order, O(1)), LinkedHashSet (insertion order), TreeSet (sorted, O(log n), no null, uniqueness by compare).
- Custom elements: `equals` + `hashCode` for hash sets; `Comparable`/`Comparator` for TreeSet.
- **Queue**: `offer`/`poll`/`peek` (return special values) vs `add`/`remove`/`element` (throw). `ArrayDeque` for queue and stack.
- PriorityQueue = min-heap; only `poll()` order is sorted; use a reversed comparator for a max-heap.
- `Comparable.compareTo` (one natural order) vs `Comparator.compare` (many). Use `Integer.compare`, never subtraction.
- `Collections`: sort, reverse, shuffle, max, frequency, binarySearch (on sorted lists), unmodifiable and synchronized wrappers.